

Technologies web

HTML, CSS, JavaScript, HTTP et sécurité

- HTML décrit le contenu (le texte d'un document ou *page web*) ainsi que sa structure logique (son organisation interne).
- Un hypertexte est un ensemble de documents HTML dans lequel on peut se déplacer grâce à un système de renvois appelés hyperliens ou liens hypertextes. (visuellement, un hyperlien est souvent un texte souligné qui change d'aspect au survol, mais il peut aussi être placé sur une image ou un bouton).
- CSS décrit l'apparence du document HTML.
- JavaScript est un langage de programmation compris et interprété par la plupart des navigateurs *web*. Il permet d'(inter)agir avec la page (son contenu comme son apparence).
- HTTP est le protocole d'échange d'informations entre client *web* (navigateurs par exemple) et serveur *web*.

1. HTML

Ce langage de description fait appel à des **balises** dont les noms figurent entre *chevrons* : « < » et « > ». C'est nécessaire pour distinguer un texte ordinaire d'une instruction ayant un sens pour le navigateur, comme par exemple `<h1>`. Il existe plus de 100 balises HTML : connaître les principales est utile.

 **Remarque : comment insérer un commentaire ? ▼**

En HTML, tout texte encadré par `<!--` et `-->` sera présent dans le document mais sera absent de l'affichage.

1.1. Structure d'une page

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>Titre de l'onglet</title>
    <link href="style.css" rel="stylesheet">
    <script src="code.js"></script>
  </head>
  <body>
    <!-- contenu visible ici -->
  </body>
</html>
```

```
<!-- Indispensable -->
<!-- Indispensable : balise racine (lang bienvenu) -->
<!-- Important pour indiquer les métadonnées -->
<!-- Recommandé (nécessaire si texte pas en UTF-8) -->
<!-- Facultatif, mais fortement recommandé -->
<!-- Nécessaire s'il y a une feuille de style CSS -->
<!-- Nécessaire pour un code JavaScript externe -->

<!-- Indispensable : le contenu de la page suit -->
```

 **Remarque : ▼**

- `<head>` : dédié aux métadonnées et aux ressources externes (CSS, JS) — invisible à l'écran.
- `<body>` : contient le contenu réellement affiché.

- Les balises peuvent aussi bien être écrites en majuscules qu'en minuscules. Autrefois les majuscules primaient, ça n'est plus le cas désormais.

1.2. Balises essentielles

<code><h1>Titre</h1></code>	<code><!-- Titres de sections, de h1 à h6 --></code>
<code><p>Paragraphe</p></code>	<code><!-- Espacement avant et après le contenu --></code>
<code>
</code>	<code><!-- Force le passage à la ligne --></code>
<code>important emphase</code>	<code><!-- Visuellement : texte en gras / en italique --></code>
<code>lien</code>	<code><!-- Lien vers URL ou #ancre interne à la page --></code>
<code></code>	<code><!-- Déclaration d'une image (balise autofermante) --></code>
<code><div>Bloc générique</div></code>	<code><!-- Bloc (les div sont empilées verticalement) --></code>
<code>en ligne générique</code>	<code><!-- Bloc en ligne (reste dans le flux du texte) --></code>
<code><table id="ancre"></code>	<code><!-- Tableau, cible d'un lien, grâce à l'ancre --></code>
<code><tr></code>	<code><!-- Déclare une ligne de tableau (TR : Table Row) --></code>
<code><th>En-tête</th></code>	<code><!-- Cellule d'en-tête (TH : Table Head) --></code>
<code><th>En-tête</th></code>	
<code></tr></code>	
<code><tr></code>	<code><!-- Cellule du tableau (TD : Table Data) --></code>
<code><td>Cellule</td></code>	
<code><td>Cellule</td></code>	
<code></tr></code>	
<code></table></code>	
<code></code>	<code><!-- Déclare une liste à puces (Unordered List) --></code>
<code>Élément 1</code>	<code><!-- Élément de la liste --></code>
<code>Élément 2</code>	
<code></code>	<code><!-- ... : liste numérotée (Ordered) --></code>

Remarque : balises additionnelles ▼

Balise	Rôle
<code><header></code> / <code><footer></code>	En-tête / pied de page (ou de section)
<code><nav></code>	Bloc de navigation (liens principaux)
<code><main></code>	Contenu principal de la page (unique)
<code><section></code>	Regroupement thématique
<code><aside></code>	Contenu complémentaire / secondaire
<code><blockquote></code>	Citation longue (bloc)
<code><figure></code> / <code><figcaption></code>	Image avec légende
<code><hr></code>	Trait de séparation horizontal (<i>horizontal rule</i>)

i Attributs et valeurs

Dans le code précédent, trois balises possèdent des **attributs**, chacun ayant une **valeur** associée :

- `<a href="url">` : cette balise ouvre une ancre (*anchor* en anglais). Il y a deux types d'ancres :
 - celles qui pointent vers une autre ressource (page, fichier à télécharger) ;
 - celles qui servent à cibler un passage spécifique de la page en cours ;
 - dans tous les cas, l'attribut `href` doit être présent, et vaudra soit un chemin (ex. `href=https://linuxfr.org`), soit un repère du document préfixé du symbole `#` (ex. `href=#ancree`).
- `` : cette balise permet d'inclure une image dans le document. Il est nécessaire de préciser le chemin d'accès à ce fichier-image (chemin local sur le serveur, ou URL) : c'est le rôle de l'attribut `src` (source). Ici, l'image est située dans le *même répertoire* que le fichier HTML et se nomme `photo.jpg`.

Fournir une description *via* l'attribut `alt` est important pour l'accessibilité du document (cela aide les malvoyants et les robots logiciels).

- `<table id="ancree">` : ici, l'attribut `id` permet à la fois le ciblage par une ancre du type `<a href="#ancree">Voir le tableau</a>`, soit la manipulation par un code JavaScript de cet élément spécifique de la page.

1.3. Formulaires

Un formulaire est un élément HTML qui permet la saisie d'informations et déclenche leur traitement. C'est l'une des briques de l'interactivité des sites *web*.

```
<form method="post" action="traitement.php">
  <label for="nom">Nom :</label>
  <input type="text" id="nom" name="nom">
  <input type="submit" value="Envoyer">
</form>
```

Attribut	Rôle
<code>method</code>	<code>get</code> (données visibles) ou <code>post</code> (données cachées) <u>requête HTTP</u>
<code>action</code>	URL de la page qui traitera les données
<code>name</code>	Nom du champ transmis au serveur

Plus en détails

- **Balise `<form>`** : elle déclare le formulaire.
 - `method="post"` : les données sont envoyées dans le corps de la requête HTTP, invisibles dans l'URL.
 - `method="get"` : les données apparaîtront dans l'URL : `traitement.php?nom=Alice&age=17` (par exemple).
 - `action="traitement.php"` : l'URL vers laquelle le formulaire envoie ses données. C'est une page côté serveur (ici un script écrit en langage PHP) qui les reçoit et les traite. Sans cet attribut, les données sont envoyées à l'URL de la page courante.

- **Balise `<input>`** : elle introduit une zone de saisie.
 - `type` : nature du champ (`text`, `number`, `email`, `password`, `checkbox`, `radio`, `submit` ...). Le navigateur adapte le clavier (sur mobile) et peut valider le format.
 - `name` : c'est l'attribut crucial. Il détermine le *nom du paramètre transmis* au serveur. Sans `name`, le champ n'est pas envoyé.
 - `id` : sert uniquement à lier le champ à son `<label>` via `for`, et à le cibler en JavaScript (non transmis au serveur).
 - `placeholder` : texte indicatif grisé dans le champ (il disparaît dès le début d'une saisie).
- **Balise `<label>`** : sert à préciser la nature du contenu qui devra être saisi par l'utilisateur.

`for="nom"` : lie le label à l' `<input id="nom">`. Un clic sur le label donne le focus au champ (important pour l'accessibilité).

Ce qui est transmis au serveur

Au clic sur le bouton « Envoyer », si l'internaute a saisi `Alice` dans le champ `nom` et `7` dans `age` :

- **Avec `method="get"`** : le navigateur forge une requête `GET` vers `traitement.php?nom=Alice&age=17` (un script PHP local au serveur car `https://...` est absent). Les paramètres sont dans l'URL, visibles de tous.
- **Avec `method="post"`** : le navigateur envoie une requête `POST` vers `traitement.php`, avec les paramètres *dans le corps* de la requête. Cela signifie que les données figurent dans les bits transmis au serveur, mais que l'internaute n'en a pas connaissance (comme dans la plupart des communications réseau : on en constate l'effet, on ne voit pas le flux de bits). Le script de traitement recevra l'association champ-valeur `nom=Alice` **et** `age=17`. Ces informations sont invisibles dans l'URL, mais elles ne sont pas pour autant chiffrées (il faut passer par [HTTPS](#) pour cela).

2. CSS : mise en forme

HTML définit le **fond**, CSS définit la **forme**.

Remarque : comment insérer un commentaire ? ▼

En CSS, tout texte encadré par `/*` et `*/` figurera dans le document mais sera ignoré.

2.1. Lier CSS à HTML

On peut intégrer du code CSS *directement* dans une page HTML, en déclarant **dans la section** `<head>` de la page les règles CSS entre deux balises `<style>...</style>`. Mais c'est une **mauvaise pratique**, car le code CSS est alors dupliqué sur chaque page HTML.

Mieux vaut écrire le code CSS dans un fichier à part (par exemple `style.css`), qui peut alors être *partagé* entre plusieurs documents HTML. On l'indique (toujours dans le `<head>`) par :

```
<link href="style.css" rel="stylesheet">  <!-- méthode recommandée, dans le <head> -->
```

La maintenance du code CSS (corrections de bugs, évolutions) s'en trouve ainsi grandement facilitée.

2.2. Sélecteurs et règle CSS

Une règle CSS peut s'appliquer à :

- un *nom* de balise spécifique. *Toutes* les balises identiques seront affectées par une règle de ce type ;
- une *classe*, qui concernera *plusieurs* balises (potentiellement différentes, mais toujours explicitement associées à cette classe) ;
- à une balise *spécifique* (unique).

```
/* Sélecteur de nom de balise */
p { color: red; } /* Tout texte dans un <p> sera rouge */
/* Idem avec PLUSIEURS balises simultanément sélectionnées */
p, div { color: blue; } /* Tout texte dans <p> OU <div> sera en bleu */
/* Sélecteur de classe : préfixe « . » obligatoire */
.test { color: green; } /* .test dans le CSS, mais class="test" en HTML */
/* Sélecteur d'id : préfixe « # » obligatoire */
#titre { color: pink; } /* #titre dans le CSS, mais id="titre" en HTML */
```

Le code HTML (utile) associé pourra ressembler à :

```
<h1 id="titre">Un titre</h1>
<p>
  Début du paragraphe
  <div class="test">Première DIV (avec classe 'test')</div>
  <div>Deuxième DIV (sans classe)</div>
</p>
```

2.3. Spécificité (priorité décroissante) et cascade

À la section précédente, la couleur (du texte) est définie plusieurs fois, dans le CSS :

- pour la balise `<p>` (deux fois) ;
- et pour la balise `<div>` (une fois « en général », une autre pour les balises `<div>` de classe `test`).

Quel sera alors le rendu du code précédent ? Réponse ci-contre !

Un titre

Début du paragraphe

Première DIV (avec classe 'test')

Deuxième DIV (sans classe)

Explication : la règle de plus grande spécificité !

Voici, par priorité décroissante, la façon dont les conflits de déclarations sont réglés :

- Style inline > `#id` > `.classe` > balise.
- À égalité, la règle déclarée **en dernier** l'emporte.

Exemple : explication de ce qui précède ▼

Le contenu de `<p>` voit sa couleur définie à `red`. Elle est ensuite redéfinie à `blue` avec celle des balises `<div>`. Le texte « Début de paragraphe » apparaît donc en bleu, de même que « Deuxième DIV (sans classe) ».

Le texte « Première DIV (avec classe 'test') » a une `class` définie, à laquelle correspond la couleur de texte `green` : c'est pourquoi il apparaît en vert sur la capture ci-dessus.

2.4. Quelques propriétés

```
.element {
  color: #3a1a0a;           /* Couleur du texte */
  background-color: #fff8e7; /* Couleur du fond */
  font-size: 1.2em;         /* Taille de la police */
  font-weight: bold;        /* Gras */
  text-align: center;       /* Alignement du texte */
  border: 1px solid black;   /* Bordures pour encadrer quelque chose */
  padding: 10px;            /* Espace intérieur, autour du texte */
  margin: 0 auto;          /* Espace extérieur & centrage horizontal */
}
```

3. JavaScript : interagir avec la page

JavaScript est un langage de programmation qui s'exécute **dans le navigateur** et permet de modifier la page sans la recharger. C'est ce qu'on appelle agir sur le **DOM** (*Document Object Model*) : la représentation de la page sous forme d'arbre d'objets manipulable programmatiquement.

Dans un document HTML minimal, insérer le code ci-dessous puis observer le résultat dans un navigateur.

```
<p id="message">Texte initial</p>
<button onclick="changerTexte()">Cliquer</button>

<script>
  function changerTexte() {
    document.getElementById("message").textContent = "Texte modifié !";
  }
</script>
```

- Explications :** le script précédent accède à l'élément dont l'`id` est "message" et modifie son contenu. Plus précisément :
- `document.getElementById("id")` : récupère un élément par son `id` (`id` à adapter selon les besoins).
 - `.textContent` : modifie le texte d'un élément (on reconnaît la notation objet).
 - `onclick` : déclenche une fonction lors d'un clic (événement).

4. Requêtes HTTP

Le **client** (navigateur) et le **serveur** communiquent via le protocole **HTTP**. Le client envoie une commande au serveur : on parle de **requête** (parfois de *méthode*). Le serveur lui renvoie alors une **réponse**. Il y a plusieurs types de requêtes : seules `GET` et

POST nous intéressent ici.

4.1. GET vs POST

Ce sont deux requêtes permettant au client *web* d'envoyer des informations au serveur *web* pour traitement.

	GET	POST
Paramètres	Visibles dans l'URL (?nom=Alice)	Dans le corps de la requête
Usage	Consultation, données non sensibles	Données confidentielles, formulaires

 **Piège !**

Sans HTTPS, les données émises par une requête POST sont **tout aussi lisibles** sur le réseau que celles d'une requête GET . Seul HTTPS garantit la confidentialité.

4.2. Codes de réponse

Chaque requête forgée par un client *web* à destination d'un serveur *web* entraîne l'envoi par ce dernier d'un code de réponse.

Code	Signification
200	Succès
301 / 302	Redirection (la ressource a changé d'adresse, ou renvoi d'une adresse en http:// vers la version sécurisée https://)
403	Accès interdit
404	Ressource introuvable
500	Erreur serveur

5. Sécurisation des communications avec HTTPS

5.1. Chiffrement symétrique

Les deux parties partagent la **même clef** pour chiffrer et déchiffrer.

- ✔ Rapide, efficace pour de grands volumes de données.
- ❌ Problème : comment transmettre cette clef de façon sécurisée ? Si elle est interceptée, *toute communication reposant sur cette clef* est compromise !

5.2. Chiffrement asymétrique

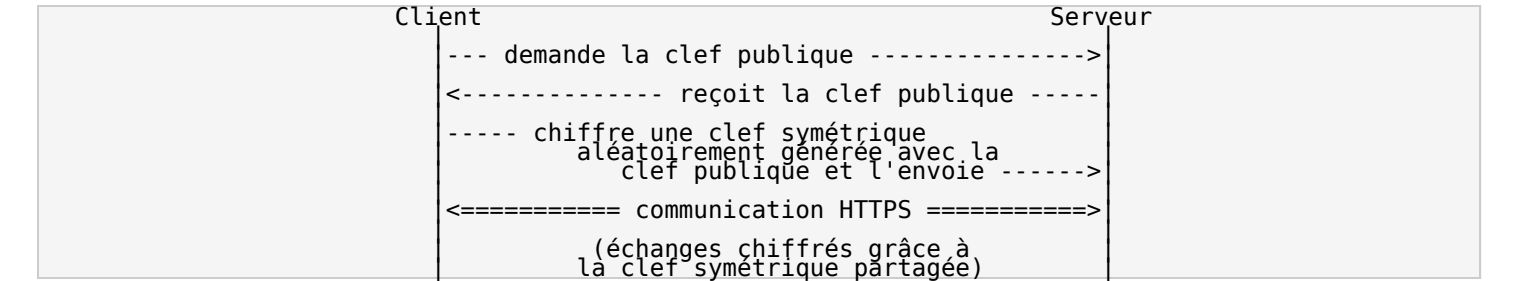
Chaque partie possède une paire de clefs : une **clef publique** (diffusable librement) et une **clef privée** (secrète). Ce qui est chiffré avec la clef publique ne peut être déchiffré qu'avec la clef privée correspondante.

-  Résout le problème de l'échange de clef.
-  Lent : peu adapté au chiffrement de gros volumes de données.

5.3. HTTPS : le meilleur des deux

HTTPS combine les deux approches :

1. Le client récupère la **clef publique** du serveur.
2. Il l'utilise pour **chiffrer et transmettre une clef symétrique** au serveur.
3. Seul le serveur peut déchiffrer ce message (grâce à sa clef privée).
4. Les deux parties partagent désormais une clef symétrique secrète : la communication chiffrée peut commencer.



Remarque : pour la culture ▼

- Le protocole qui met en œuvre ce mécanisme s'appelle **TLS** (*Transport Layer Security*), anciennement SSL.
- La clef symétrique est régulièrement renégociée, de sorte à éviter qu'un attaquant doté d'une grande puissance de calcul ne puisse la déchiffrer avant la fin de la communication.
- La négociation détaillée de ce protocole n'est pas au programme.